



Natural Language Processing (CS60075)

Assignment 1 Report

Submitted to: Prof. Saptarshi Ghosh

Submitted By: Vinamra Choudhary (25BM6JP58)

Prakhar Goyal (25BM6JP36)

1. Introduction

This assignment uses statistical n-gram language models, namely Unigram, Bigram, and Trigram, using Maximum Likelihood Estimation (MLE). Two smoothing techniques were used to address data sparsity: Laplace (Add-1) and Good-Turing.

The language models were trained using the provided Wikipedia training data and tested using three different test datasets:

- Wikipedia (in-domain)
- Arxiv abstracts (cross-domain)
- Financial news (cross-domain)

The performance of the language models was measured using Perplexity, a metric of how well they can predict text. The lower the Perplexity, the better the performance of a language model.

2. Preprocessing and Vocabulary

Construction

The following preprocessing steps were applied to the training and test datasets:

- Lowercasing all text
- Removal of non-ASCII characters
- Removal of punctuation using regular expressions
- Normalisation of numeric patterns (years, percentages, numbers)
- Lemmatisation using WordNetLemmatizer
- Removal of tokens of length ≤ 1
- Addition of sentence boundary markers `<s>` and `</s>`
- Replacement of out-of-vocabulary (OOV) words with `<UNK>`

2.1 Vocabulary Filtering

To minimise sparsity and enhance computational efficiency, n-grams that occurred in at least 1% of the documents were retained. Document frequency, and not token frequency, was used in the construction of the vocabulary.

This significantly reduced sparsity and improved computational efficiency.

3. Model Formulation

3.1 Maximum Likelihood Estimation (MLE)

Unigram Model

$$P(w) = \frac{C(w)}{N}$$

Where:

- $C(w)$ is the count of word w
 - N is total number of tokens
-

Bigram Model

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Trigram Model

$$P(w_i | w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$$

4. Smoothing Techniques

4.1 Laplace Smoothing

Laplace smoothing adds 1 to all counts to avoid zero probabilities:

$$P_{Laplace}(w_i | h) = \frac{C(h, w_i) + 1}{C(h) + V}$$

Where:

- V is vocabulary size.

4.2 Good–Turing Smoothing

Good–Turing smoothing adjusts counts using:

$$c^* = (c + 1) \frac{N_{c+1}}{N_c}$$

Where:

- N_c is the number of n-grams that occur exactly c times.

Conditional probabilities were computed per history:

$$P(w \mid h) = \frac{c^*(h, w)}{C(h)}$$

5. Time Taken

Section	Time (seconds)
Preprocessing	~204 sec
Model Building & Estimation	~175 sec
Evaluation	~ 49 sec

Total runtime was within acceptable limits for the dataset size.

6. Perplexity Results

6.1 Laplace Smoothing

Model	Wiki	ArXiv	Finance
Unigram	538.411	496.321	778.520
Bigram	666.219	1396.687	1788.870
Trigram	2527.946	4124.343	6866.820

6.2 Good–Turing Smoothing

Model	Wiki	ArXiv	Finance
Unigram	553.814	511.462	809.299
Bigram	563.129	1238.896	1589.676
Trigram	2035.927	3799.962	5250.520

7. Analytical Evaluation of Domain Shift

7.1 Behavior of the Unigram Model

The unigram model assumes independence between words and only predicts marginal word probabilities as follows:

$$P(w_1, w_2, \dots, w_n) = \prod_i P(w_i)$$

However, since this model ignores word context, its performance relies solely on vocabulary distribution similarity between the training and test data.

The perplexity of the test data on Arxiv is slightly lower than that of the test data on Wikipedia. This means that the marginal word distribution of the test data on Arxiv abstracts is similar to that of the training data on Wikipedia. The reason for this is as follows. Scientific articles, especially in the field of computer science, frequently use a limited set of repetitive vocabulary, such as “model,” “data,” “method,” “analysis,” and so forth. This means there is a high level of word overlap between Wikipedia articles and the Arxiv dataset.

This leads us to an important point:

Cross-domain data does not necessarily imply higher perplexity for unigram models, since they model only first-order statistics.

7.2 Bigram and Trigram Models Under Domain Shift

In contrast to the unigram model, the bigram and trigram models include the following contextual dependencies:

$$P(w_i \mid w_{i-1}), P(w_i \mid w_{i-2}, w_{i-1})$$

The results indicate a significant increase in perplexity:

- Finance has the highest perplexity values.
- ArXiv shows moderate degradation.

- Wikipedia, the in-domain model, has the lowest perplexity values in all cases.

This supports the observation that the contextual patterns differ significantly across domains. Although the vocabulary may be similar, the transition probabilities differ substantially across encyclopedic, scientific, and financial content.

For example:

- Financial content has specific phrase structures and vocabulary.
- Scientific content has strict syntactic conventions.
- Wikipedia shows greater variability in syntax.

The higher-order models detect these contextual regularities; thus, they are more sensitive to changes in the distribution across domains.

7.3 Sensitivity to Model Order

An important observation is that perplexity degradation increases with n-gram order.

- Unigram: minimal sensitivity
- Bigram: moderate sensitivity
- Trigram: strongest degradation

This behavior reflects a fundamental trade-off:

Model Order	Context Awareness	Robustness to Domain Shift
Unigram	Low	High
Bigram	Medium	Moderate
Trigram	High	Low

This is because higher-order models offer better in-domain modelling owing to richer contextual representations, but their generalisation across domains is poor, especially when structural patterns change.

It is, therefore, expected that the trigram models will perform worse under domain change, as instructed by the assignment.

8. Comparative Analysis of Smoothing Techniques

8.1 Laplace vs Good–Turing

The Laplace smoothing method evenly distributes the probabilities of all possible events by adding 1 to the count of each event. Although this method is straightforward and consistent, it overestimates the probabilities of less common events and underestimates the probabilities of more common ones.

The Good-Turing smoothing method takes into account the frequency of the frequency distribution:

$$c^* = (c + 1) \frac{N_{c+1}}{N_c}$$

This technique redistributes probability mass between infrequent and frequent n-grams more intelligently.

From the results:

- Good-Turing tends to perform slightly better than the Laplace method for bigram and trigram models.
- Improvements are minor but consistent.
- For the unigrams, the differences are minor because the sparsity issue is less critical.

This validates the claim that the Good-Turing method handles sparsity better, especially in higher models.

9. Computational Efficiency and Design

9.1 Decisions

Several design decisions were implemented to improve both efficiency and statistical robustness:

1. **Document Frequency Filtering (1% Rule)**

Only n-grams appearing in at least 1% of training documents were retained. This significantly reduced vocabulary size and improved computational efficiency.

2. **Rare Word Replacement using <UNK>**

Words below the frequency threshold were mapped to <UNK>, reducing sparsity and preventing unreliable probability estimates.

3. **Numeric Normalization**

Numeric expressions were normalized into semantic categories such as:

- num
- year
- percent

This prevented vocabulary explosion caused by distinct numeric tokens (e.g., different years or percentages). By collapsing numeric variants into shared categories, we reduced sparsity and improved both memory efficiency and statistical reliability.

4. Efficient Data Structures

- Counter for frequency counting
- defaultdict for conditional grouping
- History-based grouping for bigram/trigram probabilities

5. Modular Design

Clear separation between preprocessing, counting, smoothing, and evaluation improved readability and maintainability.

These optimizations reduced runtime while maintaining correctness.

10. Theoretical Insights

The experimental results illustrate key principles of statistical language modeling:

- Language models trained on a specific domain encode both lexical and structural regularities.
- Lexical similarity alone does not guarantee contextual similarity.
- Higher-order n-grams capture richer structure but generalize poorly across domains.
- Proper smoothing is critical for handling data sparsity, particularly in higher-order models.

The observed results are consistent with theoretical expectations in statistical NLP.

11. Conclusion

This assignment successfully implements and evaluates n-gram language models using MLE, Laplace smoothing, and Good–Turing smoothing. Experimental findings demonstrate:

- Unigram models are relatively domain-agnostic due to reliance on marginal word distributions.
- Bigram and trigram models show significant sensitivity to domain shift.
- Good–Turing smoothing provides modest but meaningful improvements over Laplace in higher-order models.
- Model complexity improves in-domain performance but reduces robustness under distributional shift.

Overall, the results highlight the trade-off between contextual modeling power and cross-domain generalization, a fundamental theme in language modeling research.